SAS Innovate 2026: PROC SUPERLEARNER MODEL

Why use PROC SUPERLEANER ?

Machine learning is a branch of artificial intelligence that allows computers to learn from data through iterations. Different types of algorithms are trained on large datasets, and the system then uses data to make predictions, identify patterns, and make decisions leading to improved business performance over time. This introduces a new procedure in SAS Viya called PROC SUPERLEARNER. This procedure constructs a predictive model that combines multiple predictions from machine learning models.

Demo Overview (~45 mins demo)

Demonstration Steps

1. From the ribbon at the bottom of the screen, launch Google Chrome. When the browser opens, select **SAS Landing** from the bookmarks bar or from the link on the page.
2. If the user ID and password are not pre-filled, do the following:
   - Enter **student** in the **User ID** field.
   - Enter **Metadata0** in the **Password** field.

Note: Use caution when you enter the user ID and password because values can be case sensitive.

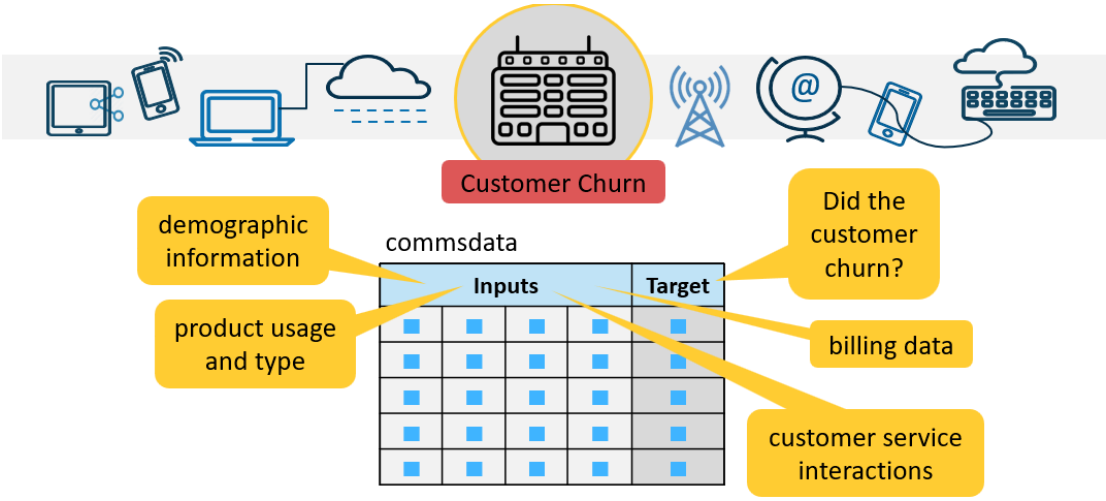
3. Click **Sign In**.
4. In the Assumable Groups window, select **Yes**. The SAS Landing home page appears.
5. From **SAS Landing**, you can access SAS Viya products.
   In the upper left corner click the **Applications** menu and select **SAS Studio: Develop Code and Flow**.
6. On the right of the interface navigate to the SAS Server tab, click on the tab. Navigate to the **"Files"** directory, under the File directory click **"Courses"** folder and navigate to the **"PROC_SUPERLEARNER"** folder.
7. Open the **PRE_PROCESSING_SUPERLEARNER.sas** file. This file is responsible for handling the data preprocessing and formatting for our data. You may take time in between the **"PROC_SUPERLEARNER_DEMO"** to run this bonus material.

Folder Path : Files → Home → Courses → PROC_SUPERLEARNER
Preprocessing_SUPERLEARNER.sas file (Run lines 15-86) → PROC SUPERLEARNER MODEL.sas

Data Preparation

In this demonstration, we will create a new project in Model Studio. This is the project that is used throughout this 45-minute demonstration. During this demo, we will load the **Comms** dataset into memory from a local drive in the virtual lab environment.

Demo Scenario:
A fictitious telecommunications company has a problem with customer's churn. When existing customers or subscribers stop doing business or otherwise end a relationship with a company, this is known as churn, and this is bad for the bottom line. It is more costly to attract new customers than to keep existing customers. The data contains information about customer behavior, it consists of demographic information, product usages, product types, billing data, and customer service interactions.



In this stage we will explore the data and give some high-level overview of the data. We first must select a target variable. Target in this case will be the "BAD". Whether the customer defaulted on their loan or not. This is binary target, which gives us a sense of direction of how to start this process.

Load data In-Memory

In this section, we will load our "COMMS" dataset into our SAS Viya environment using the LIBNAME statement. This statement allows us to connect a user-defined shortcut to a

physical folder on our computers. It points to an existing directory to make dataset within it directly accessible in SAS.

```
libname ps '/home/student/Courses/PROC_SUPERLEARNER/data';
libname out '/home/student/Courses/PROC_SUPERLEARNER/output';
```

We create to LIBNAME statement “ps” & “out”, this is where we will store our data that we load into our environment and where we store our output datasets.

Data STEP: Loading data into CAS Server

```
/***    LOAD DATA IN-MEMORY     *******/
data ps.telecomms_clean;
    set ps.commsdata;
run;
```

In this code, we name our new data table as “**ps.telecomms_clean”** . We call our data telecommunication dataset from our data file location as **“ps.commsdata”**.

Data Exploration

Explore by using PROC DESCRIBE procedure to provide a summary statistic of the data. Provide the shape of the data, check for missing data and the distribution of data as well.

```
proc means data= ps.telecomms_clean N Mean std min max skewness kurtosis;
run;
```

The PROC MEANS procedure provides a data summarization tool to compute descriptive statistics for variables across all observations and within groups of observations. PROC MEANS does the following:

- Calculates descriptive statistics based on moments

- Estimate quantiles, which includes the median
- Calculates confidence limits for the mean
- Identifies extreme values
- Performs a *t* test

Feature Engineering / Data Preprocessing

In this section, we will handle some feature engineering, from what we gather so far from our summary statistic is that within the data we have some variables that have negative minimum values for our customer data. It is possible that for some industry, the negative values are used to indicate account credit for a customer. For our demonstration we will handle the negative values by setting all negative values (< 0) to zero. By removing the negative values from our data it will ensure that we have no outliers that could drastically affect our predictions.

```
/**  Handling the negative values **/;
data ps.tele_comms_clean;
    set ps.telecomm_clean;

 /**  The original data with negative values   **/;
 array old_limits(*) tot_mb_data_roam_curr seconds_of_data_norm lifetime_value
bill_data_usg_m03
 bill_data_usg_m06 voice_tot_bill_mou_curr mb_data_usg_roamm01 mb_data_usg_roamm02
mb_data_usg_roamm03
 mb_data_usg_m01 mb_data_usg_m02 mb_data_usg_m03 calls_total calls_in_pk calls_out_pk
calls_in_offpk calls_out_offpk
 mb_data_ndist_mo6m data_device_age mou_onnet_pct_MOM mou_total_pct_MOM;

 array new_limits(*) tot_mb_data_roam_curr seconds_of_data_norm lifetime_value
bill_data_usg_m03
 bill_data_usg_m06 voice_tot_bill_mou_curr mb_data_usg_roamm01 mb_data_usg_roamm02
mb_data_usg_roamm03
 mb_data_usg_m01 mb_data_usg_m02 mb_data_usg_m03 calls_total calls_in_pk calls_out_pk
calls_in_offpk calls_out_offpk
 mb_data_ndist_mo6m data_device_age mou_onnet_pct_MOM mou_total_pct_MOM;

 /**  Apply rule to set negative values to zero  **/;
 do i=1 to dim(old_limits);

 /**  If the old value is a negative value then we will set the new value to 0  **/
```

```
    if old_limits[i] < 0 then new_limits[i] = 0;
    /** If the value is greater than 0, then we keep the original value **/
    else old_limits[i] = new_limits[i];
    end;
run;
```

The MEANS Procedure

Variable	Label	N	Mean	Std Dev	Minimum	Maximum	Skewness	Kurtosis
Customer_ID	Primary Key	56557	1871721.32	1214488.00	471.0000000	3999922.00	0.0549030	-1.2549491
churn	Churn Flag	56557	0.1213289	0.3265120	0	1.0000000	2.3195744	3.3805450
lifetime_value	Lifetime Value	56557	5281.53	5068.84	-14006.00	60740.20	1.8543235	5.7694165
avg_arpu_3m	3M Avg Revenue per User	55437	60.2948845	22.8771098	0	160.3761848	0.8185036	2.3206915
acct_age	Account Tenure	56557	45.1726555	12.9064363	18.0000000	165.0000000	0.6693952	1.7918378
billing_cycle	Billing Cycle	56557	6.6233357	3.1898376	1.0000000	12.0000000	-0.0289219	-0.8397392
nbr_contracts_ltd	Total Number Contracts Lifetime	56557	4.1616747	2.6373436	1.0000000	16.6453080	0.9679281	1.2641342
rfm_score	Account Ranking (RFM Score)	56557	221.9940768	82.0648094	111.0000000	333.0000000	0.000131570	-1.4704193
Est_HH_Income	Estimated HH Income	56557	31734.13	13799.92	0	263400.00	1.4026823	7.9566850
cs_med_home_value	Census Area Median Home Value Index	56434	2.1796190	1.5152938	0	9.9900000	1.9978241	5.5990919
cs_pct_home_owner	Census Area Percent Home Owner	56434	0.5777558	0.2602454	0	0.9900000	-0.4732351	-0.7986969
cs_ttl_pop	Census Area Total Population	55239	27601.26	18987.27	7.0000000	114124.00	1.0247724	1.4580546
cs_hispanic	Census Area Hispanic	55169	12.7517981	18.8474017	0.0400000	98.9100000	2.2799542	4.9767429
cs_caucasian	Census Area Caucasian	55235	69.6051136	27.2503133	0.3900000	99.8700000	-1.0102982	-0.0922614
cs_afr_amer	Census Area African-American	54695	11.5994990	18.4501180	0.0100000	100.0000000	2.5168167	6.3704496
cs_other	Census Area Other	55222	6.1808817	8.7397736	0.0200000	99.2600000	4.6557528	29.8402548
cs_ttl_urban	Census Area Total Urban	55239	79.1021812	31.4095466	0	100.0000000	-1.5721755	1.2131983
cs_ttl_rural	Census Area Total Rural	55239	20.8978213	31.4095475	0	100.0000000	1.5721752	1.2131974
cs_ttl_male	Census Area Total Males	55239	49.0903170	2.4321788	0.6900000	97.1300000	3.8606207	47.0060458
cs_ttl_female	Census Area Total Female	55239	50.9096834	2.4321785	2.8700000	99.3100000	-3.8606221	47.0060668
cs_ttl_hhlds	Census Area Total Households	55232	10088.67	6623.03	2.0000000	61774.00	0.8334123	1.3085916
cs_ttl_mdage	Census Area Median Age	55239	35.3785568	5.1257309	14.0000000	75.0000000	0.7833713	4.9415544
forecast_region	Forecasted Region Key	56557	45.9955090	26.8456454	0	92.0000000	0.000147342	-1.2004130
mb_inclplan	Plan Data MB	56557	849.8417526	439.6062976	250.0000000	1500.00	0.2337719	-1.2799281
ever_days_over_plan	Total Days Over Plan	56557	13.7506586	15.8381629	0	142.0000000	1.6871174	4.1333637
ever_times_over_plan	Total Times Over Plan	56557	2.5303499	2.4527833	0	26.0000000	1.0691828	1.5822748
data_device_age	Avg Age of Devices on Plan	52835	924.1027463	617.5700425	-50.0000000	2600.19	0.4496267	-0.3694745
equip_age	Handset Age	56557	20.0226851	13.1547543	0	49.0000000	-0.2059709	-1.2566574
mfg_apple	Own Apple	56557	0.4552398	0.4979969	0	1.0000000	0.1797671	-1.9677534
mfg_samsung	Own Samsung	56557	0.2319076	0.4220541	0	1.0000000	1.2704621	-0.3859397
mfg_htc	Own HTC	56557	0.0884771	0.2839900	0	1.0000000	2.8982527	6.4000952
mfg_motorola	Own Motorola	56557	0.0439910	0.2050769	0	1.0000000	4.4473562	17.7796063
mfg_lg	Own LG	56557	0.1142387	0.3181038	0	1.0000000	2.4254655	3.8830202
mfg_nokia	Own Nokia	56557	0.0611242	0.2395601	0	1.0000000	3.6641445	11.4263587
delinq_indicator	Delinquent Indicator	56557	0.1265272	0.3883281	0	4.0000000	3.5727607	15.5104632
times_delinq	Consecutive Mths Delinquent	56557	0.3919232	0.7062139	0	6.0000000	1.8791267	3.2411619
count_of_suspensions_6m	Times Suspended Last 6M	56557	0.4426266	0.9443556	0	5.0000000	2.6743535	7.0331033
avg_days_susp	Days Suspended Last 6M	56557	3.4713735	3.8312731	0	62.0000000	1.5937222	5.0680748
calls_total	Total Calls Curr	56557	727.7480985	615.5984674	-1837.35	7949.91	1.7104540	4.7739613
calls_in_pk	Calls Incoming Peak	56557	235.4593419	241.5422011	-1816.00	3487.33	1.5665171	7.1118747
calls_in_offpk	Calls Incoming Off-Peak	56557	388.6468614	406.1405144	-1410.35	4640.24	1.8185824	5.6826097
calls_out_offpk	Calls Outgoing Off-Peak	56557	31.3590419	30.1620496	-109.0000000	331.0000000	1.3056653	3.8581909
calls_out_pk	Calls Outgoing Peak	56557	72.2828533	83.4204312	-498.0000000	1603.67	2.5904642	14.5607265
mou_total_pct_MOM	Minutes Total Pct Change Month over Month	47453	0.0572678	1.8850544	-29.0000000	92.3000000	3.4333704	150.4456332
mou_onnet_pct_MOM	Minutes On Network Pct Change Month over Month	29812	-0.2595395	3.8853929	-45.0000000	124.7272727	2.8059664	91.5615366
mou_roam_pct_MOM	Minutes Roaming Pct Change Month over Month	21245	0.3590290	10.9338986	-83.0000000	319.7000000	2.7611304	71.8312286
voice_tot_bill_mou_curr	Total Voice Billed Minutes of Use	56557	5836.33	5639.84	-10912.74	72726.56	2.1484758	7.3929925
tot_voice_chrgs_curr	Total Voice Charges	55437	34.4301344	11.7843762	0	93.8935079	0.3894804	0.5010001
tot_drpd_pr1	Number of Dropped Calls 1 Mth Prior	50754	2.6227095	3.6448450	0	34.0000000	3.2757163	15.3655455
bill_data_usg_m03	3M Avg Billed Data Usage	56557	1864.91	1634.51	-13678.00	40767.10	1.3974299	13.7883885
bill_data_usg_m06	6M Avg Billed Data Usage	56557	1869.14	1418.91	-10874.30	16813.60	1.3000284	5.0747826
bill_data_usg_m09	9M Avg Billed Data Usage	56557	1919.54	1862.27	0	84386.50	5.6973783	113.3316954
mb_data_usg_m01	MB Data Usage 1 Mth Prior	56557	1697.20	1549.21	-2425.00	40568.65	2.0378909	15.4829147
mb_data_usg_m02	MB Data Usage 2 Mths Prior	56557	1698.65	1547.89	-2171.06	40761.34	2.0176794	15.3247900
mb_data_usg_m03	MB Data Usage 3 Mths Prior	56557	1696.25	1556.10	-1621.00	40784.23	2.0524790	15.6742987
mb_data_ndist_mo6m	6M Avg Billed Data Usage Normally Distributed	56442	0.0626561	4.1430941	-92.4717301	87.7572409	0.1290002	18.7767314
mb_data_usg_roamm01	MB Data Usage Roam 1 Mth Prior	56557	133.8719876	573.2160210	-4358.00	11400.00	1.2315465	12.1567053
mb_data_usg_roamm02	MB Data Usage Roam 2 Mths Prior	56557	125.7287692	603.0714436	-7033.00	18727.00	2.2428314	39.4702820
mb_data_usg_roamm03	MB Data Usage Roam 3 Mths Prior	56557	135.2706827	649.9658573	-5984.00	12720.00	1.3668832	16.5920467
tot_mb_data_curr	Total MB of Data Usage	56557	2361.74	2289.80	-10786.47	66288.07	4.6153732	49.8621406
tot_mb_data_roam_curr	Total MB of Roam Data Usage	56557	477.4561657	2887.31	-60034.48	66288.07	2.1106411	32.9572663
bill_data_usg_tot	Total Billed Data Usage	56557	44.6595470	11.0659616	17.0000000	82.0000000	0.4048293	-0.1327397

We also noticed that from the Data Exploration, we saw high levels skewness beginning with MB_Data_Usg. We create a DATA step where we call the variables MB_Data_Usg_M04

through MB_Data_Usg_M09. We will handle this by taking the LOG of the of these six variables to ensure evenly distributed data.

MB_Data_Usg_M04	MB of Data Usage Month 4	56557	159.3068586	381.1479077	0	14606.00	9.8151533	179.7305071
MB_Data_Usg_M05	MB of Data Usage Month 5	56557	142.7952685	471.5577885	0	24707.00	13.6121741	315.4085581
MB_Data_Usg_M06	MB of Data Usage Month 6	56557	230.5485970	718.7863878	0	29676.00	15.2432110	360.4407283
MB_Data_Usg_M07	MB of Data Usage Month 7	56557	94.2739537	259.8390503	0	13672.00	17.6345171	495.6025665
MB_Data_Usg_M08	MB of Data Usage Month 8	56557	109.5912442	348.7336436	0	16297.00	16.9031377	467.9810534
MB_Data_Usg_M09	MB of Data Usage Month 9	55660	95.9823751	218.3546743	0	8869.00	7.8438804	131.2951273

```
data ps.telecomms_clean;
    set ps.telecomms;
    /* Original variables */
    array old_vars(*) MB_Data_Usg_M04 MB_Data_Usg_M05 MB_Data_Usg_M06
        MB_Data_Usg_M07 MB_Data_Usg_M08 MB_Data_Usg_M09;

    /* New variables */
    array new_vars(*) LOG_MB_Data_Usg_M04 LOG_MB_Data_Usg_M05
        LOG_MB_Data_Usg_M06 LOG_MB_Data_Usg_M07 LOG_MB_Data_Usg_M08
        LOG_MB_Data_Usg_M09;

    /* Apply rule */
    do i=1 to dim(old_vars);
        /* If negative → log transform */
        if old_vars[i] < 0 then new_vars[i]=log(abs(old_vars[i]));

        /* If zero or positive → keep original value */
        else new_vars[i]=old_vars[i];
    end;

    /* Drop variables (outside loops) */
    drop i upsell_xsell city city_lat city_long data_usage_amt
        mou_onnet_6m_normal mou_roam_6m_normal region_lat region_long state_lat
        state_long tweedie_adjusted zip_lat zip_long zipcode_primary;
run;

data ps.tele_comms_clean;
    set ps.telecomms_clean;

    drop MB_Data_Usg_M04 MB_Data_Usg_M05 MB_Data_Usg_M06 MB_Data_Usg_M07
        MB_Data_Usg_M08 MB_Data_Usg_M09;
run;
```

```
/*************************************************************/
proc format;
    value churn 0='Stay' 1='Churn';
run;

data ps.telecomms_clean;
    set ps.telecomms_clean;
    format churn churn.;
run;
```

This section of code handles our extreme outliers where some of the values are negative numbers, we utilize array index on our original variables to create our new values to replace the values. The following data step handles the skewness; we take the log of our six variables to improve the distribution of the data. After we have successfully executed the DATA step, we will then drop the original MB_Data_Usg variables.

Data Partition

The next step of this demonstration is to partition our data into our training and validation set of analysis; we utilize our target variable for the partition. Remember we are working with binary target variable with customers ending business with the fictitious telecommunication company.

```
/***** DATA PARTITION:SPLITTING OUR DATA INTO A 70/30 SPLIT  *************/;
proc  partition data=ps.telecomms_clean samppct=70 seed=12345 partind;
    by churn;
    output out=ps.telecomms_part copyvars=(_all_);
run;
```

The PARTITION Procedure

Stratified Sampling Frequency			
Index	Churn Flag	Number of Obs	Number of Samples
0	Stay	49695	34787
1	Churn	6862	4803

Output Tables			
Library	Name	Number of Rows	Number of Columns
OUT	TELECOMMS_PART	56557	114

In the following code, we begin with the partitioned dataset created using the PROC PARTITION procedure. A random seed is specified, and the target variable churn is defined at an interval level. We split our data into 70 training and 30 percent validation, we implement a random seed, we specify we want to include the partition indicator within our dataset. The specficy our target variable of churn. We produce an output saved to our LIBREF "ps" labeled as telecoms_part. Finally, we copy over all variables and run the PROC PARTITION procedure.

Format Churn

In this section we change the format of the variable churn, we do this by specifying whether a person stays or churns(ends business with the telecommunication company). The reason for this is that the model will only observe the probability of the lowest level of the TARGET alphanumerically. If we leave the churn outcomes as 1 and 0 the model will only observe the observations of a customer continuing business with the telecommunication company, but the goal is to provide predictive probabilities of a customer you have ended business with the telecommunications company.

```
/** Modify the Partition Indicator: 0 =Training,1 = Validation  ****/
proc format;
    value _partind_ 0='Training' 1='Validation';
run;

/**Applying format to the variable churn**/

data out.telecomms_part;
    set out.telecomms_part;
    format _partind_ _partind_.;
run;
```

We use the **PROC FORMAT** procedure we select the value that we want to format as "_partind_". We format **"0"** as Training and **"1"** as Validation. Next, we apply the format to the variable **"_PartInd_"** to include the new format.

Macro-Variable Inputs\

In this section, we provide a list of input variables that we suggest using for the model. This input variable can vary, but we suggest using the listed inputs for simplicity of the demonstration.

```
/***  Macro-Variable for Inputs for Model  ****/
%let VI_inputs=pymts_late_ltd ever_days_over_plan mou_total_pct_MOM
      curr_days_susp  delinq_indicator  times_delinq  mou_roam_pct_MOM
     equip_age
    bill_data_usg_tot nbr_contracts_ltd rfm_score Est_HH_Income
       cs_med_home_value   cs_pct_home_owner   cs_ttl_pop   cs_hispanic
     cs_caucasian
      cs_afr_amer   cs_other   cs_ttl_urban   cs_ttl_rural   cs_ttl_male
     cs_ttl_female
    cs_ttl_hhlds cs_ttl_mdage forecast_region mb_inclplan;
```

The macro variable **&VI_inputs** are used to represent the input variables, which are specified at an interval level.

Building the PROC SUPERLEARNER Model: BASELEARNER Algorithms

```
/***************BUILDING THE PROC SUPERLEARNER MODEL **********/;


proc superlearner data=out.telecomms_part(where=(_PartInd_= 1 ))
seed=23456;
    target churn / level=nominal;
    input &VI_inputs / level=interval;
    baselearner 'GB' gradboost(leafsize=50 numbin=100);
    baselearner 'Forest' forest( maxdepth=15 ntrees=100
inbagfraction=0.6
        minleafsize=5);
    baselearner 'DT' treesplit;
```

```
    baselearner 'Logistic' logselect(link=logit selection=stepwise);
    crossvalidation kfold=3;
    output out=ps.predict2_output2 copyvar=churn;
    store out=out.scoreData2;
run;
```

We build the remainder of the model using the BASELEARNER statements. In this model, we utilize the light gradient boosting, the Forest, the Decision Tree, and the Logistic Regression model. We use more baselearners and specify the hyperparameters for each of the models. By default, each baselearner has its own set of hyperparameters that you can adjust to optimize the model. The Cross-validation is performed using a 3-fold approach, and the prediction results are written to the output table out.predict_output. We use the store statement to save and create our score data labeled as out.scoreData2 to be used against new or existing holdout data for model accuracy.

The SUPERLEARNER Procedure

Model Information	
Data Source	TELECOMMS_PART
Target Variable	churn
Target Variable Type	NOMINAL
Number of Folds	3
Number of Base Learners	4
Loss Function	Log Loss
Meta-learning Method	Convex-constrained Binomial Likelihood Maximization
Random Number Seed	23456

Number of Observations Read	39590
Number of Observations Used for Training	14309
Number of Observations Used for Scoring	39590

Probability modeled is churn = Churn.

Super Learner Model Coefficients			
Name	Model Type	Coefficient	Cross-Validated Risk
LGB	Light Gradient Boosting Machine	0	0.31661
Forest	Forest	0.95481	0.20683
DT	Decision Tree	0	0.12654
Logistic	Logistic Regression	0.04519	0.22610

Output Tables			
Library	Name	Number of Rows	Number of Columns
OUT	SCOREDATA2	1	2
PS	PREDICT2_OUTPUT2	39590	2

In the above figure, The SUPERLEARNER procedure was applied to the TELECOMMS_PART dataset to predict the target variable churn using 3-fold cross-validation and **four base learners: light gradient boosting (LGB), the Forest, the Decision Tree (DT), and the Logistic Regression model)**. The ensemble used quadratic loss with a convex constrained least squares meta-learning method to combine model predictions. From the results of the SUPERLEARNER model coefficient, we see that the Light Gradient Boosting (LGB) did not contribute to the final ensemble, receiving the largest cross-validated risk of 0.31 and the Decision Tree with the second highest cross-validated risk of 0.22. The Forest model was the overall contributor with a coefficient of 0.95 with a cross validated risk of 0.12 and the same for Logistic Regression. The final predictions were saved to the **"PS.PREDICT_OUTPUT"** table.

Score Output for Validation Data

```
/*******Score Validation from the Previous Model************/
proc superlearner data=out.telecomms_part restore=out.score_model;
    output out=out.scoreData2 learnerpred copyvars=(churn _partind_);
run;

proc print data=out.scoredata2(obs=10);
run;
```

Scoring Information	
Store	SCORE_MODEL
Scoring Table	TELECOMMS_PART
Training Table	TELECOMMS_PART
Target Variable	churn

Number of Observations Read	56557
Number of Observations Used for Scoring	21245

Probability modeled is churn = Churn.

Output Tables			
Library	Name	Number of Rows	Number of Columns
OUT	SCOREDATA2	56557	6

Obs	P_churnChurn	LGB	Forest	Logistic	churn	_PartInd_
1	0.10672	0.13723	0.11750	0.10151	Stay	Training
2	0.12226	0.21489	0.13931	0.11403	Churn	Training
3	0.23784	0.33300	0.41263	0.15348	Stay	Training
4	0.13883	0.05764	0.06998	0.17206	Stay	Validation
5	0.69125	0.52426	0.62931	0.72114	Stay	Validation
6	0.28641	0.07740	0.12394	0.36484	Stay	Training
7	0.30565	0.25151	0.18530	0.36374	Stay	Validation
8	0.13104	0.16170	0.12164	0.13558	Stay	Validation
9	0.09110	0.03948	0.04781	0.11200	Stay	Training
10	0.08534	0.04111	0.05022	0.10230	Stay	Validation

In this section of the code, we take score data from the PROC SUPERLEARNER model, to use for comparison on our holdout or validation set to test the model accuracy. We call the PROC SUPERLEARNER statement, but we will use the procedure restore to call the score data from the PROC SUPERLEARNER Model. We create and output table labeled as **"OUT.SCOREDATA2"**

Assessing The Event of Churn

```
/**** ASSESSING EVENT CHURN by the Partition  ***/;
ods output ROCInfo=out.roc_table2;

/* <----- This is the ROC Curve table output */
proc assess data=out.scoredata2;
    var P_churnChurn;
    target churn / event="Churn" level=nominal;
```

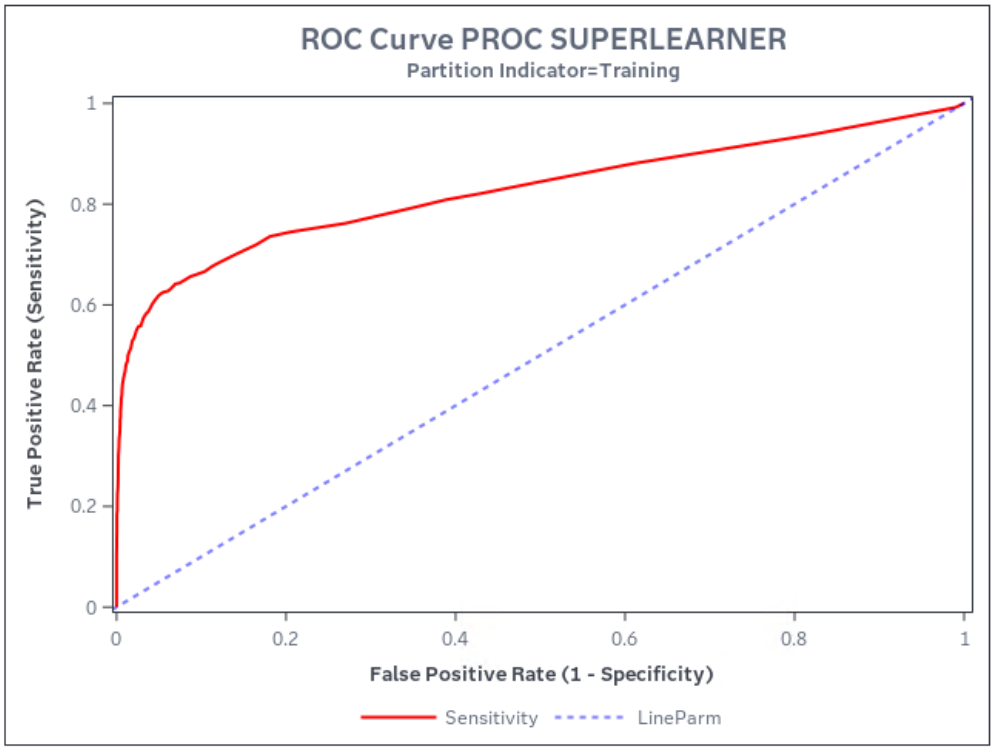
```
    by _partind_ ;
    /** Output ROC table **/
run;
ods output close;
```

The **PROC ASSESS** allows us to evaluate, compare, and validate the performance of predictive models. It helps to compute model assessment statistics such as lift, gain, ROC information by both our validation and training sets.

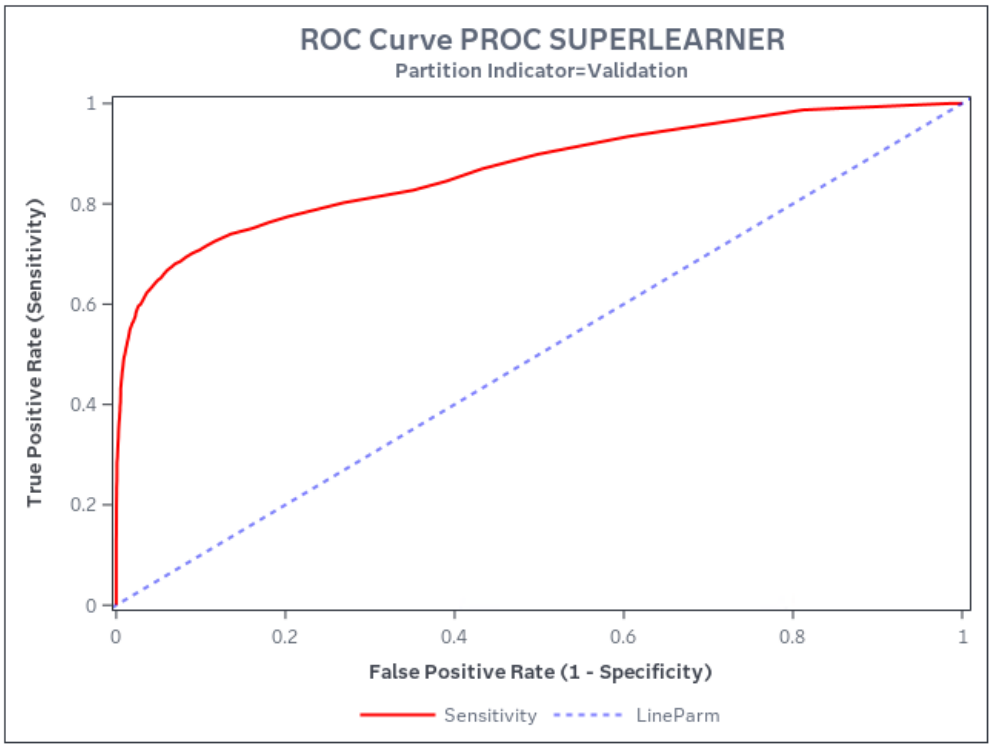
```
proc print data=out.roc_table2;
run;
```

In the above code, we use the PROC ASSESS procedure to assess the model performance and obtain the ROC Data Information. For this procedure we specify the information we want to assess. We name the data **"OUT.ROC_TABLE2"** that will provide us with ROC Curve Information need to create our ROC Curve using the PROC SGPLOT. The variable we call is **"P_churnChurn",** we call the target variable **"churn"** with the event as **"Churn"** with a level of nominal which is customers who have the probability of ending business with the telecommunication company. We separate them by the **"_PartInd_"** indicator to provide use with the ROC curve for both the Training and Validation set.

```
proc sgplot data=out.roc_table2;
    series x=FPR y=Sensitivity / lineattrs=(thickness=2 color=RED);
    by _partind_;
    lineparm x=0 y=0 slope=1 / transparency=0.5
lineattrs=(pattern=shortdash
        color=BLUE); /* diagonal */
    xaxis label="False Positive Rate (1 - Specificity)" min=0 max=1;
    yaxis label="True Positive Rate (Sensitivity)" min=0 max=1;
    title "ROC Curve PROC SUPERLEARNER";
run;
```



The Figure above provides the performance of our Training set for the PROC SUPERLEARNER model.

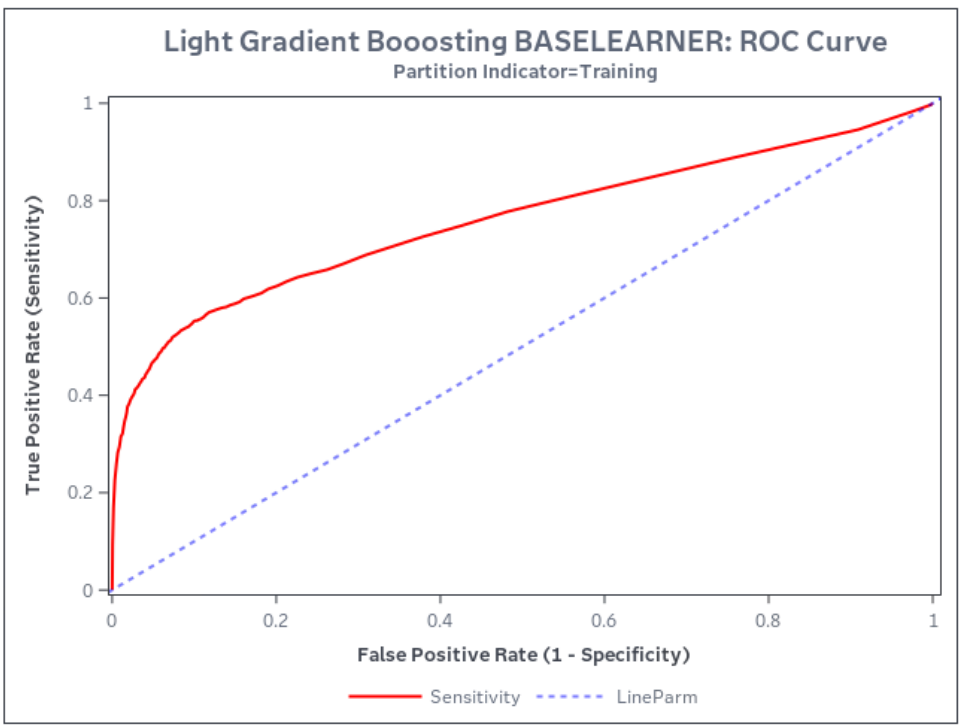


The Figure above provides the performance of our Validation set for the PROC SUPERLEARNER model.

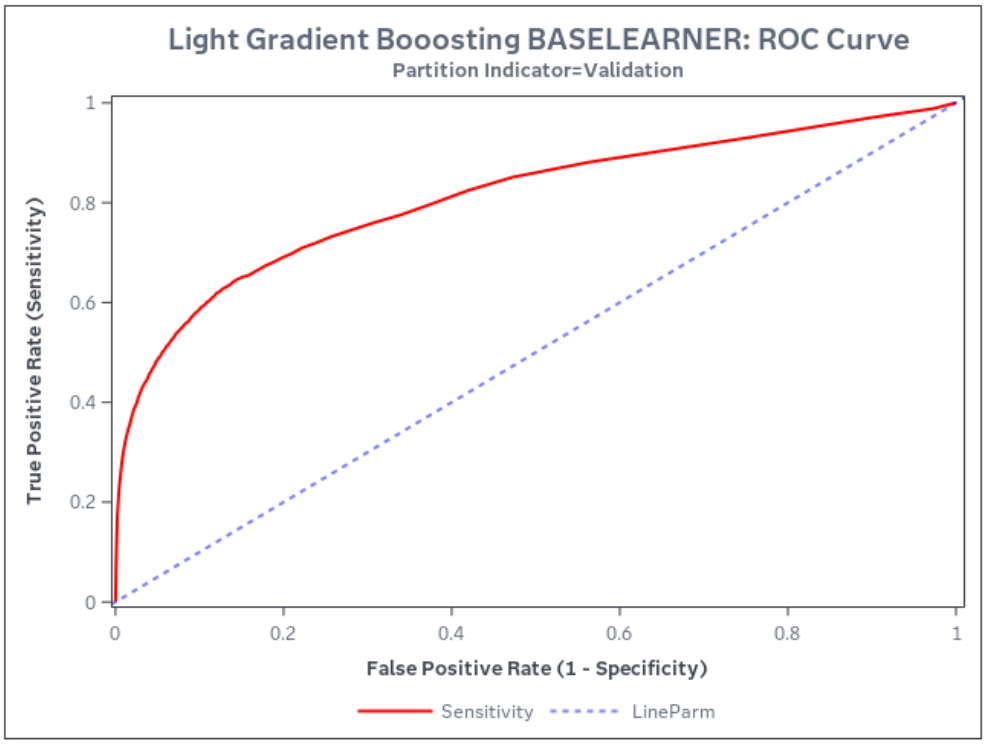
ROC Curve: BASELEARNERS

```
/*********************************************************************
****/


proc sgplot data=out.roc_table;
    series x=FPR y=Sensitivity / lineattrs=(thickness=2 color=RED);
    by _partind_;
    lineparm x=0 y=0 slope=1 / transparency=0.5
lineattrs=(pattern=shortdash
        color=BLUE); /* diagonal */
    xaxis label="False Positive Rate (1 - Specificity)" min=0 max=1;
    yaxis label="True Positive Rate (Sensitivity)" min=0 max=1;
    title "PROC SUPERLEARNER: ROC Curve";
run;

/********    ROC PROC SUPER VS BASELEARNERS   *************/;

/*********************************************************/



/***** ROC for Light-Gradient Boosting Model ****/
ods output ROCInfo=out.roc_table_LGB;

/* <----- This is the ROC Curve table output */
proc assess data=out.score_data;
    var LGB;
    target churn / event="Churn" level=nominal;
    by _partind_ ;
    /** Output ROC table **/
run;
ods output close;
```

```
proc print data=out.roc_table_LGB;
run;

proc sgplot data=out.roc_table_LGB;
    series x=FPR y=Sensitivity / lineattrs=(thickness=2 color=RED);
    by _partind_;
    lineparm x=0 y=0 slope=1 / transparency=0.5
lineattrs=(pattern=shortdash
        color=BLUE); /* diagonal */
    xaxis label="False Positive Rate (1 - Specificity)" min=0 max=1;
    yaxis label="True Positive Rate (Sensitivity)" min=0 max=1;
    title "Light Gradient Booosting BASELEARNER: ROC Curve";
run;
```



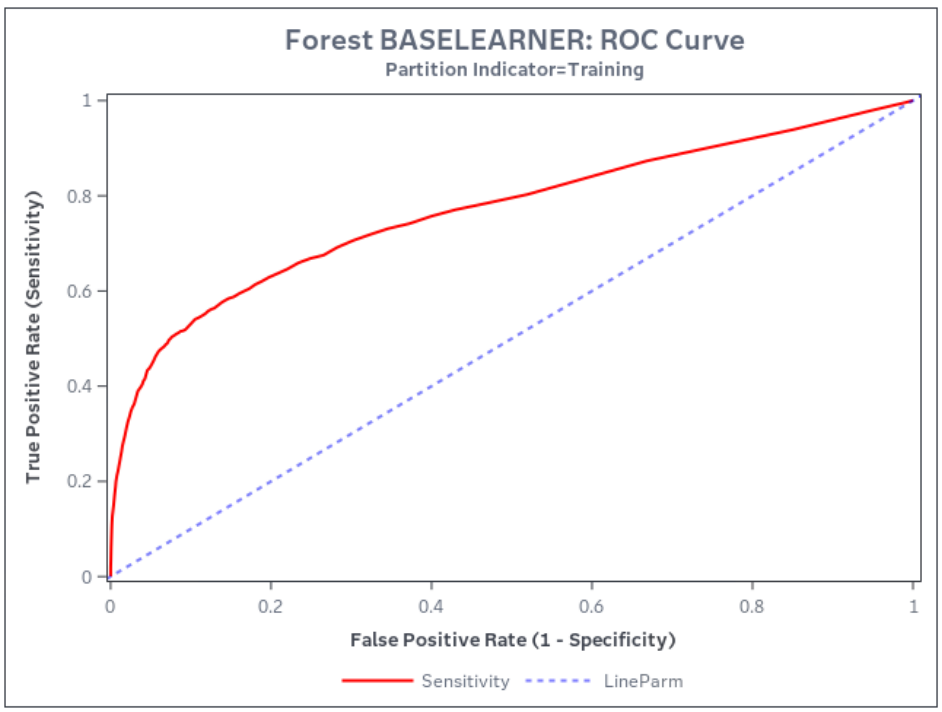
The Figure above provides the performance of our Training set for the Gradient Boosting model.



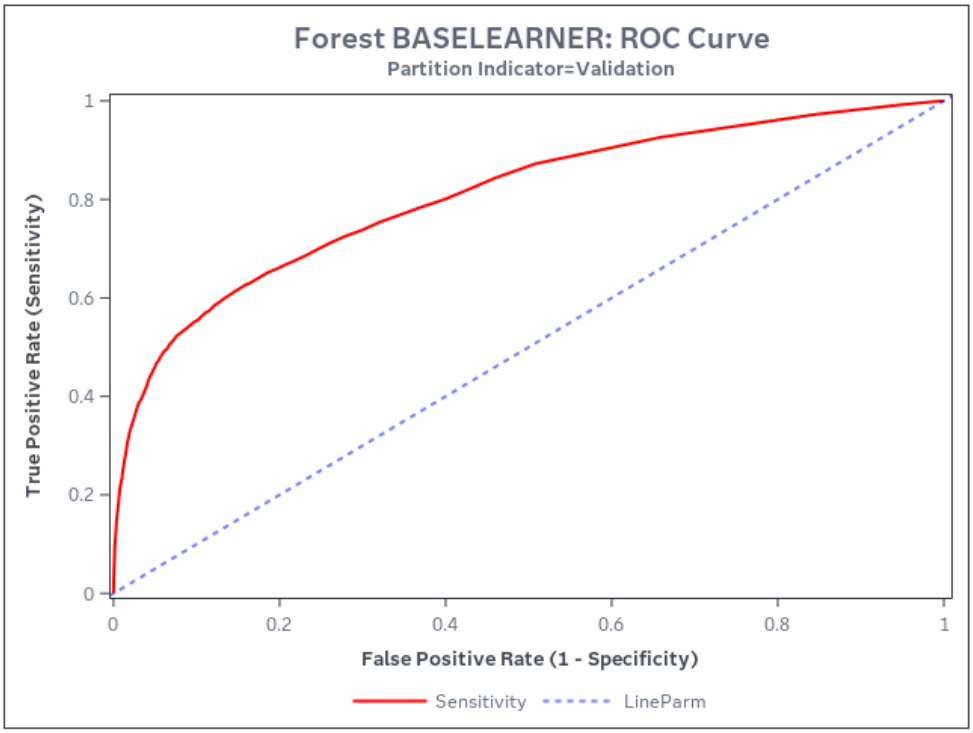
The Figure above provides the performance of our Validation set for the Gradient Boosting model.

```
/**************    ROC Curve for Forest Model    ********************/

/***************    ROC Curve Information    *************************/
ods output ROCInfo=out.roc_table_Forest;

/* <----- This is the ROC Curve table output */
proc assess data=out.score_data;
    var Forest;
    target churn / event="Churn" level=nominal;
    by _partind_ ;
    /** Output ROC table **/
run;
ods output close;

proc print data=out.roc_table_Forest;
run;
```

```
proc sgplot data=out.roc_table_Forest;
    series x=FPR y=Sensitivity / lineattrs=(thickness=2 color=RED);
    by _partind_;
    lineparm x=0 y=0 slope=1 / transparency=0.5
lineattrs=(pattern=shortdash
        color=BLUE); /* diagonal */
    xaxis label="False Positive Rate (1 - Specificity)" min=0 max=1;
    yaxis label="True Positive Rate (Sensitivity)" min=0 max=1;
    title "Forest BASELEARNER: ROC Curve";
run;
```



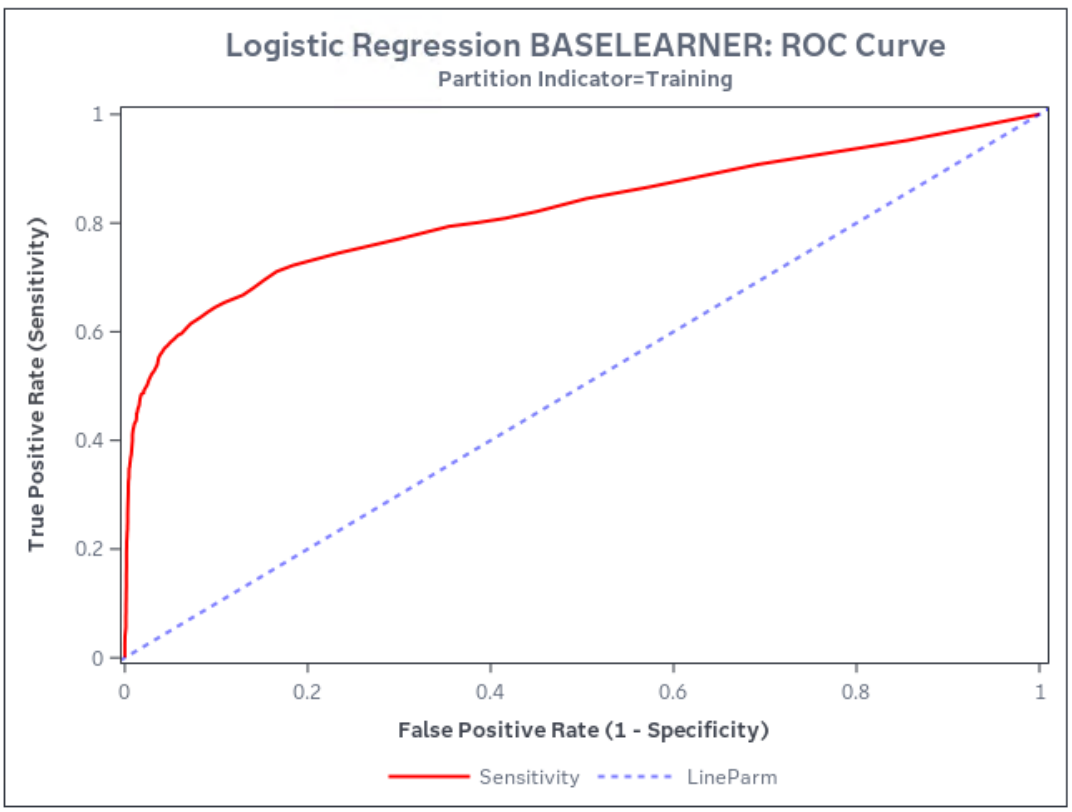
The Figure above provides the performance of our Training set for the Forest model.



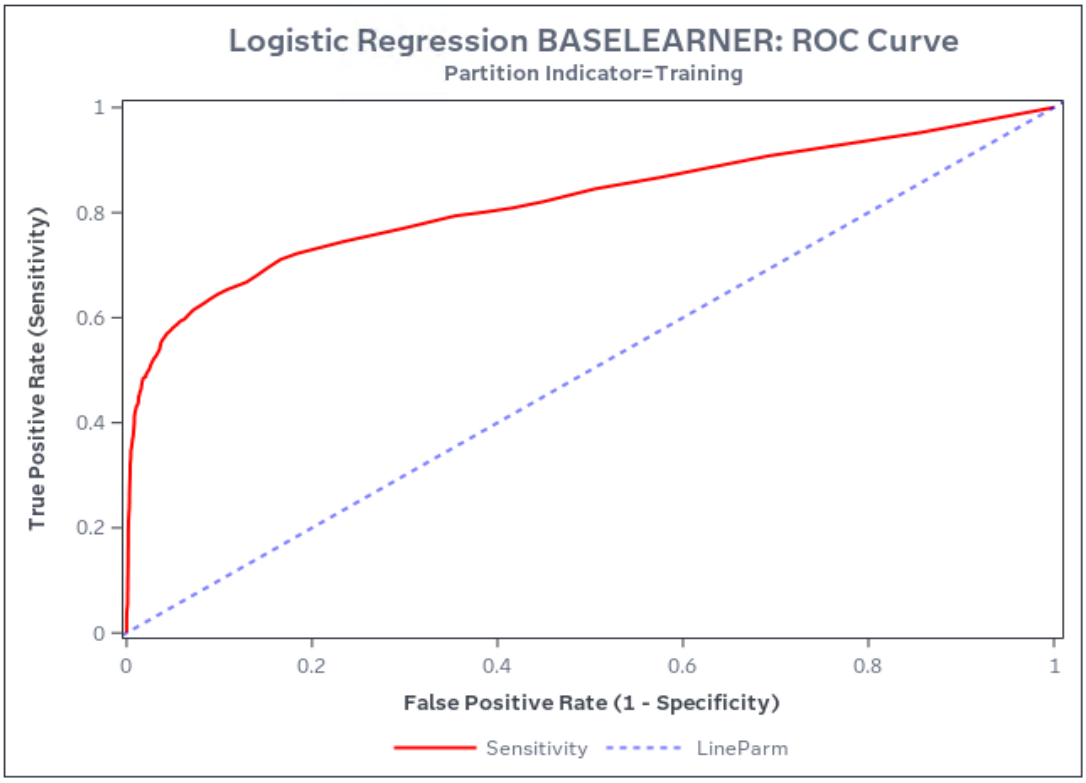
The Figure above provides the performance of our Validation set for the Forest model.

```
/***ROC CURVE for Logistic Regression Model**/


/************    ROC Curve Information   *****************/
ods output ROCInfo=out.roc_table_Logistic;

/* <----- This is the ROC Curve table output */
proc assess data=out.score_data;
    var Logistic;
    target churn / event="Churn" level=nominal;
    by _partind_ ;

run;
ods output close;

proc print data=out.roc_table_Logistic;
run;
```

```
proc sgplot data=out.roc_table_Logistic;
    series x=FPR y=Sensitivity / lineattrs=(thickness=2 color=RED);
    by _partind_;
    lineparm x=0 y=0 slope=1 / transparency=0.5
lineattrs=(pattern=shortdash
        color=BLUE); /* diagonal */
    xaxis label="False Positive Rate (1 - Specificity)" min=0 max=1;
    yaxis label="True Positive Rate (Sensitivity)" min=0 max=1;
    title "Logistic Regression BASELEARNER: ROC Curve";
run;
```



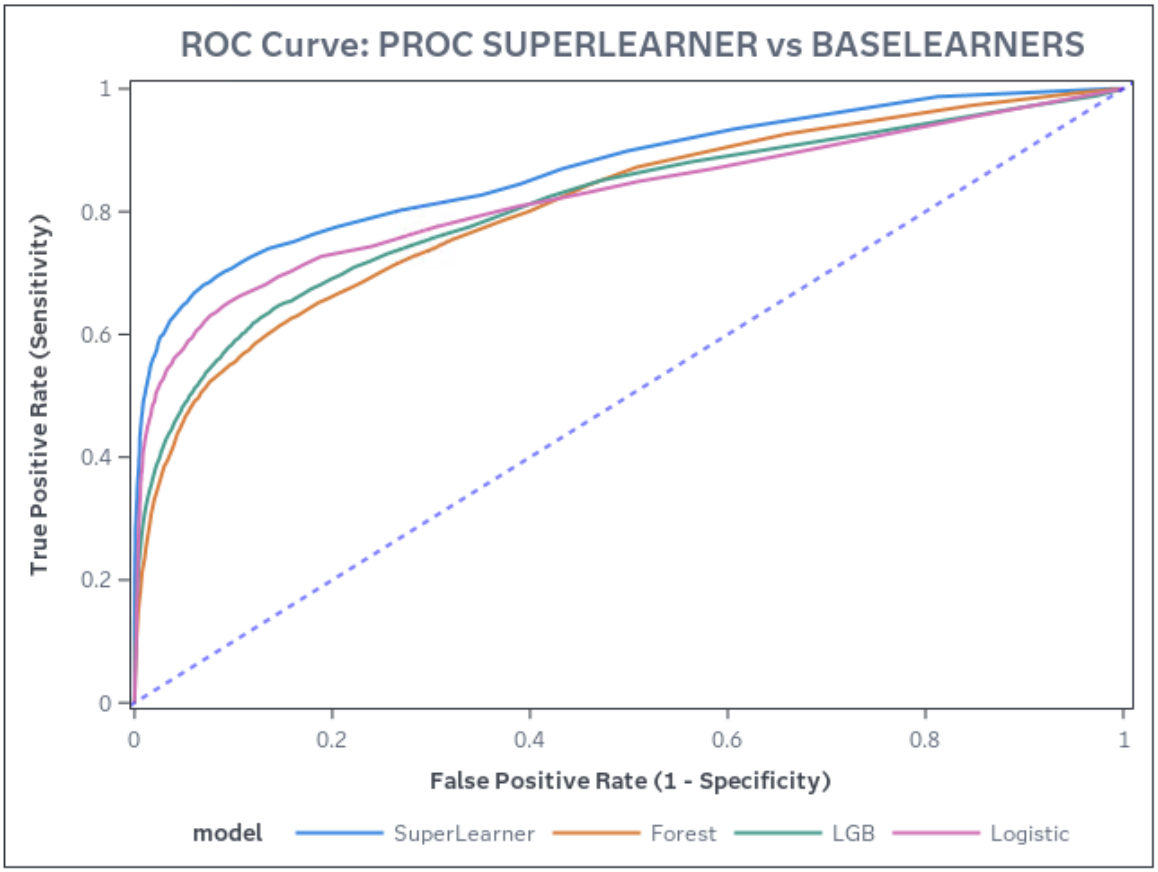
The Figure above provides the performance of our Training set for the Logistic Regression model.



The Figure above provides the performance of our Validation set for the Logistic Regression model.

```
/*************************************************************/

/************  Create a Data step that combines all ROC tables
together **************/
data out.combined_roc_data;
    set out.roc_table out.roc_table_forest out.roc_table_lgb
out.roc_table_Logistic;
    if Variable='Forest' then model='Forest';
    else if Variable='LGB' then model='LGB';
    else if Variable='Logistic' then model='Logistic';
    else if Variable='Superlearner' then model='Superlearner';
run;

proc sgplot data=out.combined_roc_data(where=(_partind_=1));
    series x=FPR y=Sensitivity / group=model;
        /*lineattrs=(thickness=2 color=RED);*/
```

```
    lineparm x=0 y=0 slope=1 / transparency=0.5
lineattrs=(pattern=shortdash
        color=BLUE); /* diagonal */
    xaxis label="False Positive Rate (1 - Specificity)" min=0 max=1;
    yaxis label="True Positive Rate (Sensitivity)" min=0 max=1;
    title "ROC Curve: PROC SUPERLEARNER vs BASELEARNERS";
run;
```



In the figure above, we have combined the ROC table reports from each model including the overall performance of the PROC SUPERLEARNER. We overlay each model into a single ROC CURVE to show the performance of each model. We notice that the SUPERLEARNER model outperforms all the single model's performance. This ROC Curve plot shows how the PROC SUPERLEARNER takes the input predictions of each BASELEARNER to create an overall SUPERLEARNER model.

Optional code: PROC HPSPLIT

The PROC HPSPLIT is high-performance procedure that builds tree-based statistical models for classification and regression. The procedure has the capabilities to produce the classification trees for categorical response, and regression trees for continuous response.

```
/***********************    Variable Importance********************/;
proc hpsplit data=ps.telecomms_part maxdepth=8;
    class churn;
    model churn=&num_inputs;
run;
```

Full Code

```
/*********************************   LIBNAME STATEMENT     **************************/;
libname ps '/home/student/Courses/PROC_SUPERLEARNER/data';
libname out '/home/student/Courses/PROC_SUPERLEARNER/output';
/*********************************     LOAD DATA IN-MEMORY     *********************/;
data ps.telecomms;
    set ps.commsdata;
run;

/************************************* SUMMARY STATISTIC   ***************************/;
proc means data= ps.telecomms N Mean std min max skewness kurtosis descending ;
run;
/***************************** DATA ENGINEERING / FEATURE EXTRACTION *******************/;
;
/**  Handling the negative values **/;

data ps.tele_comms;
    set ps.telecomms;

 /**  The original data with negative values   **/;

 array old_limits(*) tot_mb_data_roam_curr seconds_of_data_norm lifetime_value bill_data_usg_m03
 bill_data_usg_m06 voice_tot_bill_mou_curr mb_data_usg_roamm01 mb_data_usg_roamm02 mb_data_usg_roamm03
 mb_data_usg_m01 mb_data_usg_m02 mb_data_usg_m03 calls_total calls_in_pk calls_out_pk calls_in_offpk
calls_out_offpk
 mb_data_ndist_mo6m data_device_age mou_onnet_pct_MOM mou_total_pct_MOM;

 array new_limits(*) tot_mb_data_roam_curr seconds_of_data_norm lifetime_value bill_data_usg_m03
 bill_data_usg_m06 voice_tot_bill_mou_curr mb_data_usg_roamm01 mb_data_usg_roamm02 mb_data_usg_roamm03
 mb_data_usg_m01 mb_data_usg_m02 mb_data_usg_m03 calls_total calls_in_pk calls_out_pk calls_in_offpk
calls_out_offpk
 mb_data_ndist_mo6m data_device_age mou_onnet_pct_MOM mou_total_pct_MOM;

 /**  Apply rule to set negative values to zero  **/;
 do i=1 to dim(old_limits);

 /**  If the old value is a negative value then we will set the new value to 0  **/
    if old_limits[i] < 0 then new_limits[i] = 0;

    /** If the value is greater than 0, then we keep the original value **/
    else old_limits[i] = new_limits[i];

    end;
 run;
/***********    PART 2 : TAKE THE LOG OF THE MB_DATA_Usg VARIABLES ***************/;
data ps.telecomms;
    set ps.telecomms;
```

```
    /* Original variables */
    array old_vars(*) MB_Data_Usg_M04 MB_Data_Usg_M05 MB_Data_Usg_M06
        MB_Data_Usg_M07 MB_Data_Usg_M08 MB_Data_Usg_M09;

    /* New variables */
    array new_vars(*) LOG_MB_Data_Usg_M04 LOG_MB_Data_Usg_M05
        LOG_MB_Data_Usg_M06 LOG_MB_Data_Usg_M07 LOG_MB_Data_Usg_M08
        LOG_MB_Data_Usg_M09;

   /* Apply rule */
    do i=1 to dim(old_vars);

        /* If negative → log transform */
        if old_vars[i] then new_vars[i]=log(abs(old_vars[i]));

        /* If zero or positive → keep original value
        else new_vars[i]=old_vars[i]*/;

    end;

    /* Drop variables (outside loops) */
    drop i upsell_xsell city city_lat city_long data_usage_amt
        mou_onnet_6m_normal mou_roam_6m_normal region_lat region_long state_lat
        state_long tweedie_adjusted zip_lat zip_long zipcode_primary;
run;
data ps.telecomms;
    set ps.telecomms;

    drop MB_Data_Usg_M04 MB_Data_Usg_M05 MB_Data_Usg_M06 MB_Data_Usg_M07
        MB_Data_Usg_M08 MB_Data_Usg_M09;
run;
/********* CREATE A LET STATEMENT TO STORE VARIABLE FOR ANALYSIS**************/;
;
%let num_inputs=Customer_ID lifetime_value avg_arpu_3m acct_age billing_cycle
    nbr_contracts_ltd rfm_score Est_HH_Income cs_med_home_value
    cs_pct_home_owner cs_ttl_pop cs_hispanic cs_caucasian cs_afr_amer cs_other
    cs_ttl_urban cs_ttl_rural cs_ttl_male cs_ttl_female cs_ttl_hhlds
    cs_ttl_mdage forecast_region mb_inclplan ever_days_over_plan
    ever_times_over_plan data_device_age equip_age mfg_apple mfg_samsung mfg_htc
    mfg_motorola mfg_lg mfg_nokia delinq_indicator times_delinq
    count_of_suspensions_6m avg_days_susp calls_total calls_in_pk calls_in_offpk
    calls_out_offpk calls_out_pk mou_total_pct_MOM mou_onnet_pct_MOM
    mou_roam_pct_MOM voice_tot_bill_mou_curr tot_voice_chrgs_curr tot_drpd_pr1
    bill_data_usg_m03 bill_data_usg_m06 bill_data_usg_m09 mb_data_usg_m01
    mb_data_usg_m02 mb_data_usg_m03 mb_data_ndist_mo6m mb_data_usg_roamm01
    mb_data_usg_roamm02 mb_data_usg_roamm03 tot_mb_data_curr
    tot_mb_data_roam_curr bill_data_usg_tot tot_overage_chgs
    data_prem_chrgs_curr nbr_data_cdrs avg_data_chrgs_3m avg_data_prem_chrgs_3m
    avg_overage_chrgs_3m nbr_contacts calls_TS_acct open_tsupcomplnts
    num_tsupcomplnts unsolv_tsupcomplnt wrk_orders days_openwrkorders
    resolved_complnts calls_care_acct calls_care_3mavg_acct
    calls_care_6mavg_acct res_calls_3mavg_acct res_calls_6mavg_acct
    last_rep_sat_score network_mention service_mention price_mention times_susp
    curr_days_susp pymts_late_ltd calls_care_ltd seconds_of_data_norm
    seconds_of_data_log LOG_MB_Data_Usg_M04 LOG_MB_Data_Usg_M05
    LOG_MB_Data_Usg_M06 LOG_MB_Data_Usg_M07 LOG_MB_Data_Usg_M08
    LOG_MB_Data_Usg_M09;

proc contents data=ps.telecomms;
run;
/*****************    DATA PARTITION: SPLITTING OUR DATA INTO A 70/30 SPLIT  *****************/;
proc  partition data=ps.telecomms samppct=70 seed=12345 partind;
    by churn;
    output out=ps.telecomms_part copyvars=(_all_);
run;
/****************** DATA EXPLORATION:  Check skewness of our data   *************************/;
/* Variable Importance */;
proc hpsplit data=ps.telecomms_part maxdepth=8;
    class churn;
```

```
    model churn=&num_inputs;
run;
proc contents data=ps.telecomms_part out=vars(keep=name) noprint;
run;
/************************BUILDING THE PROC SUPERLEARNER MODEL ****************************/;
proc superlearner data=ps.telecomms_part seed=12345;
    target churn / level=interval;
    input &num_inputs / level=interval;
    baselearner 'LGB' lightgradboost(leafsize=50 numbin=100);
    baselearner 'Forest' forest( maxdepth=15 inbagfraction=0.7 minleafsize=15);
    baselearner 'GB' gradboost(learningrate=0.01 maxdepth=15 minleafsize=5);
    baselearner 'DT' treesplit( criterion=gini minleafsize=15);
    crossvalidation kfold=5;
    output out=ps.predict_output copyvar=churn;
run;
```